M02 - Janaka's Court: The False Witness

Category: Crypto (Schnorr OR-proofs / Fiat-Shamir) | **Flag:** `PINAKA{janaka_false_witness_the_question_asked_before_the_answer}`

TL;DR

The court accepts 1-of-2 Schnorr OR-proofs ("depositions"), and the target docket's two identities are hash-to-group points whose discrete logs nobody knows. The Fiat-Shamir challenge hashes only the context and the two public keys, never the commitments `t0, t1`, so it is fixed before anything is committed. That lets me simulate both branches at once: choose the challenges and responses first, then solve the verifier's equations for the commitments.

Recon

The challenge ships `params.json`, `instance.json`, the court's own module `pinaka_testimony.py`, and a README. The module's self-test confirms the specimen deposition for the enrolled docket verifies, so the proof format is clear. The target docket is `false-witness`, and its note says outright that both identities are "nullified seals" with no known exponent, so an honest proof is impossible.

```
M02_player ❯ ls -lgoh *.json *.py README.md && python pinaka_testimony.py && jq 'del(.p,.q)' params.json && j
q '.dockets["false-witness"].note, .target.win_condition' instance.json
-rw-rw-r-- 1 3.9K Oct 30 02:16 instance.json
-rw-rw-r-- 1  741 Oct 31 04:08 params.json
-rw-r--r-- 1 1.9K Nov  1 20:04 payload.json
-rw-rw-r-- 1 3.7K Oct 31 04:07 pinaka_testimony.py
-rw-rw-r-- 1 2.0K Oct 31 04:08 README.md
-rw-r--r-- 1 1.8K Nov  1 20:04 solve.py
-rw-r--r-- 1  831 Nov  1 20:04 verify.py
specimen deposition for 'enrolled-nobles' verifies: True
{
  "group": "order-q subgroup of Z_p^*",
  "g": 4,
  "ctx": "PINAKA/M02/janaka-court/false-witness/v1"
}
"Both identities are NULLIFIED seals derived by hash-to-group: no deponent holds either seal exponent, so an
honest testimony is impossible. The court's flag is released for a VALID deposition on THIS docket."
"A deposition that verifies as a 1-of-2 Schnorr OR-proof for the false-witness docket's identity pair release
s the flag."

M02_player ❯
```

A deposition is `(t0, t1, c0, c1, s0, s1)`. The verifier accepts it for identities `(Y0, Y1)` when:

```
c0 + c1 = c (mod q)          where c is the Fiat-Shamir challenge
g^s0 = t0 * Y0^c0 (mod p)
g^s1 = t1 * Y1^c1 (mod p)
```

In an honest OR-proof the prover knows one secret `x_b`, simulates the other branch, and is forced to commit to both `t` values before learning `c`. That ordering is what makes the proof sound.

What the hash actually binds

`_select_binding` picks the hash from `params["binding_mode"]`. The module includes a properly bound version (`_h_bind_full`, which hashes `t0` and `t1`), but `params.json` has no `binding_mode` key, so the verifier falls back to mode 0, `_h_bind`. That version hashes only `ctx`, `Y0` and `Y1`.

```
M02_player ❯ sed -n '/^def _h_bind(/,/^$/p;/^def _select_binding/,/^$/p' pinaka_testimony.py && grep -n -A3 '
c = _select_binding' pinaka_testimony.py
def _h_bind(ctx, Y0, Y1, q):
    m = hashlib.sha256(b"PINAKA-M02-FS")
    for lbl, v in (("ctx", ctx), ("Y0", Y0), ("Y1", Y1)):
        m.update(b"|"); m.update(lbl.encode()); m.update(b"|"); m.update(str(v).encode())
    return int(m.hexdigest(), 16) % q

def _select_binding(params, *args):
    mode = params.get("binding_mode", 0)
    if mode == 0:
        return _h_bind(args[0], args[1], args[2], args[5])
    elif mode == 1:
        return _h_bind_full(*args)
    else:
        return _h_bind_augmented(*args, params.get("epoch", 0), args[5])

58:    c = _select_binding(params, ctx, Y0, Y1, t0, t1, q)
59-    if (c0 + c1) % q != c % q:
60-        return False
61-    eq0 = pow(g, s0, p) == (t0 * pow(Y0, c0, p)) % p
--
92:    cc = _select_binding(params, ctx, Ys[0], Ys[1], t[0], t[1], q)
93-    c[b] = (cc - c[o]) % q
94-    s[b] = (r + c[b] * x) % q
95-    return {"t0": t[0], "t1": t[1], "c0": c[0], "c1": c[1], "s0": s[0], "s1": s[1]}

M02_player ❯
```

With `c` independent of the commitments, the order that gives the proof its soundness is gone. Every input to `c` is public and fixed, so I know `c` before choosing anything. The rest is algebra:

1. Pick `c0`, `s0`, `s1` at random.
2. Set `c1 = c - c0 (mod q)`, which satisfies the sum check.
3. Solve each branch equation for its commitment: `t_i = g^s_i * Y_i^(-c_i) (mod p)`.

Both branch equations then hold by construction and no secret exponent is involved. This is the "question asked before the answer" from the flag.

Forging

`solve.py` reads the shipped files, computes `c`, simulates both branches with fresh random values, checks the result with the court's own `verify()`, and writes `payload.json`. With `--offline` it stops before submitting.

```
M02_player ❯ python solve.py --offline
[*] binding_mode in params.json: absent -> 0 (weak)
[+] challenge fixed before any commitment: c = ...ae599c9b1d41c224db6a
[*] random c0, s0, s1; c1 = c - c0; t_i = g^s_i * Y_i^-c_i
[+] deposition verifies for false-witness (no seal exponent used)
[+] wrote payload.json

M02_player ❯
```

```
c = _h_bind(ctx, Y0, Y1, q)
c0, s0, s1 = (secrets.randbelow(q) for _ in range(3))
c1 = (c - c0) % q
t0 = pow(g, s0, p) * pow(Y0, -c0, p) % p   # g^s0 = t0 * Y0^c0
t1 = pow(g, s1, p) * pow(Y1, -c1, p) % p   # g^s1 = t1 * Y1^c1
dep = {"t0": t0, "t1": t1, "c0": c0, "c1": c1, "s0": s0, "s1": s1}
assert verify(P, Y0, Y1, dep)
```

Because `c0`, `s0` and `s1` are random, every run produces a different valid forgery.

Sanity check

`verify.py` runs three checks. The genuine specimen verifies for its own docket, and my forgery verifies for `false-witness` under the court's `verify()`. The same forgery fails `verify_strict()`, which uses the fully bound hash. That confirms the missing commitments in the hash are the whole bug.

```
M02_player ❯ python verify.py
specimen  -> enrolled-nobles, verify()        : True
forgery   -> false-witness,   verify()        : True
forgery   -> false-witness,   verify_strict() : False
  (verify_strict hashes t0, t1 too, so the simulated transcript no longer fits)

M02_player ❯
```

Flag

Posting `payload.json` to `/court/depose` gets the deposition admitted and returns the flag.

```
M02_player ❯ curl -s -X POST https://pinaka-m02-cyber-xo-x.vercel.app/court/depose -H 'Content-Type: applicat
ion/json' -d @payload.json | jq
{
  "docket": "false-witness",
  "flag": "PINAKA{janaka_false_witness_the_question_asked_before_the_answer}",
  "proclamation": "The court records a witness where none could stand.",
  "result": "admitted"
}

M02_player ❯
```

Fix: hash the full transcript, including the commitments: `c = H(ctx, Y0, Y1, t0, t1)`. That's exactly `_h_bind_full`, and setting `binding_mode: 1` would have closed the hole.

```
PINAKA{janaka_false_witness_the_question_asked_before_the_answer}
```